RESUMEN SEGUNDO PARCIAL

TEMAS

- Filosofía Lean - Método Kanban en el contexto del desarrollo de software
- Métricas en los 3 enfoques (Tradicional / Lean / Agile)

Unidad 3: Gestión de software como producto

- Prácticas continuas

Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

- Contenidos:
  - Conceptos generales sobre calidad.
  - Importancia de trabajar para y con Calidad. Ventajas y Desventajas.
  - Actividades relacionadas con el Aseguramiento de la Calidad del Software.
  - Principales Modelos de Calidad existentes (CMMI – SPICE – ISO).
  - Auditorías al Software: Auditorías de Proyecto y Auditorías al Grupo de Calidad.
  - Proceso de Auditorías
  - Calidad de Producto: Planificación de pruebas para el software- Niveles y tipos de pruebas para el software. Técnicas y herramientas para probar software.
  - Técnicas y Herramientas para la realización de revisiones técnicas del software.
  - Testing en ambientes Ágiles.

▼

Filosofía Lean - Método Kanban en el contexto del desarrollo de software

> Lean es una filosofía de gestión que busca maximizar el valor entregado al cliente y minimizar los desperdicios.

Su objetivo no es simplemente trabajar más rápido, sino identificar qué actividades aportan valor y cuáles consumen recursos sin contribuir al resultado.

En desarrollo de software, Lean busca evitar construir funcionalidades innecesarias, reducir esperas, mejorar el flujo del trabajo y favorecer el aprendizaje.

4.2. Los siete principios Lean

La cátedra presenta siete principios fundamentales.

Principio	Explicación
1. Eliminar desperdicios	Identificar y eliminar actividades que no aportan valor.
2. Amplificar el aprendizaje	Aprender continuamente mediante feedback y experimentación.
3. Embeber la integridad conceptual	Mantener la coherencia y calidad del producto en su conjunto.
4. Diferir compromisos	Evitar tomar decisiones irreversibles cuando todavía existe demasiada incertidumbre.
5. Dar poder al equipo	Permitir que las personas que realizan el trabajo participen en las decisiones.
6. Ver el todo	Optimizar el sistema completo y no únicamente actividades individuales.
7. Entregar lo antes posible	Obtener valor y retroalimentación rápidamente.

DEAD VEE

Aspectos importantes de estos principios

Eliminar desperdicios: no significa eliminar cualquier actividad que no sea programación. Por ejemplo, una revisión de calidad puede ser necesaria porque previene errores y retrabajo.

Amplificar el aprendizaje: en software no siempre se conoce la mejor solución desde el comienzo. Los equipos necesitan experimentar y aprender del resultado.

Diferir compromisos: no significa postergar todas las decisiones, sino esperar hasta disponer de información suficiente cuando resulte conveniente hacerlo.

Ver el todo: optimizar solamente desarrollo no sirve si, por ejemplo, Testing se transforma en un cuello de botella y los cambios quedan esperando durante semanas.

Entregar lo antes posible: favorece validar que el producto realmente genera valor antes de continuar incorporando funcionalidades.

4.3. Tipos de desperdicio

La presentación identifica dos grandes categorías.

Tipo 1 — Desperdicio necesario: actividades que no generan valor directamente, pero que son necesarias bajo las condiciones actuales. El objetivo es minimizarlas.

Tipo 2 — Puro desperdicio: actividades que no aportan valor y no son necesarias. El objetivo es evitarlas.

Los 7+1 desperdicios Lean

La PPT compara los desperdicios de manufactura con sus equivalentes en servicios basados en conocimiento, como el desarrollo de software.

Desperdicio	En desarrollo de software
Inventario	Trabajo parcialmente terminado que todavía no genera valor.
Sobreproducción	Funcionalidades adicionales que el usuario no necesita.
Procesamiento extra	Actividades, pasos o procedimientos innecesarios.
Transporte	Transferencias de conocimiento y cambios de tareas.
Esperas	Demoras por dependencias, aprobaciones o disponibilidad de recursos.
Movimiento	Cambios de contexto y tiempo de preparación para cambiar de actividad.
Defectos	Errores que producen correcciones y retrabajo.
Talento no utilizado	Desaprovechar conocimientos y capacidades de las personas.

TEM SIP DT

El último se considera el desperdicio adicional, de allí la expresión 7+1 desperdicios.

Ejemplo: un desarrollador tiene cinco funcionalidades empezadas, dos esperando una revisión y tres bloqueadas por dependencias.

Aunque haya mucho trabajo iniciado, eso no significa que se esté entregando valor.

Lean intenta reducir ese inventario de trabajo y favorecer que las funcionalidades realmente se terminen.

Método Kanban en el desarrollo de software

Kanban es un método para definir, gestionar y mejorar servicios de trabajo del conocimiento. Se basa en visualizar el trabajo, limitar lo que se inicia y **gestionar el flujo** para entregar valor de forma predecible. Parte del proceso actual: no exige reemplazar de golpe los roles ni la organización.

Valores de Kanban

Transparencia, equilibrio, colaboración, foco en el cliente, flujo, liderazgo, entendimiento, acuerdo y respeto.

Principios

Gestión del cambio (4):

1. Comenzar con lo que se hace ahora.
2. Respetar los roles y responsabilidades actuales.
3. Acordar la mejora mediante cambios evolutivos.
4. Fomentar el liderazgo en todos los niveles.

Entrega de servicios (3):

1. Comprender las necesidades y expectativas del cliente.
2. Gestionar el trabajo y permitir la autoorganización.
3. Revisar periódicamente el sistema de servicios y sus políticas.

Las seis prácticas generales

Práctica	Propósito
Visualizar	Tablero con estados, tarjetas y bloqueos.
Limitar WIP	Restringir la cantidad de tareas simultáneas en progreso.
Gestionar el flujo	Detectar esperas y cuellos de botella; favorecer la finalización.

Práctica	Propósito
Hacer explícitas las políticas	Acordar reglas visibles para admitir, priorizar y mover trabajo.
Ciclos de retroalimentación	Revisar periódicamente cómo funciona el sistema.
Mejorar colaborativamente y experimentar	Introducir cambios graduales y medir sus resultados.

WIP (Work In Progress): si Desarrollo tiene un límite de 3 y ya hay 3 tareas en curso, no se inicia otra hasta liberar capacidad. La idea es **dejar de empezar y empezar a terminar**. Saturar todos los recursos no garantiza mejores entregas.

Cuello de botella: si Desarrollo termina rápido pero Testing acumula trabajo, hay que mejorar el flujo completo, no solo programar más rápido.

Sistema Push vs. Pull

Push (empujar)	Pull (tirar)
Una etapa envía trabajo aunque la siguiente no tenga capacidad.	La etapa siguiente toma trabajo **cuando tiene capacidad disponible**.
Favorece acumulaciones.	Favorece controlar el WIP y finalizar tareas.

Ejemplo: Desarrollo termina 5 funcionalidades; Testing tiene capacidad para 2. Con Pull, Testing toma las que puede procesar.

Clases de servicio

Clase	Característica
Expreso	Urgencias; la PPT ejemplifica WIP = 1 y prioridad excepcional.
Fecha fija	Entrega con vencimiento; puede escalar a Expreso si peligra el plazo.
Estándar	Trabajo habitual; respeta WIP y priorización (puede usar FIFO).
Intangible	Beneficio que no se percibe de inmediato; sigue políticas acordadas.

Son **políticas de ejemplo**, no reglas universales.

Cadencias

Reunión	Frecuencia ilustrativa	Para qué
Team Kanban Meeting	Diaria	Revisar flujo, bloqueos y estado de los elementos.
Replenishment Meeting	Semanal o a demanda	Seleccionar nuevo trabajo.
Retrospective	Quincenal o mensual	Proponer mejoras del sistema.

Para el parcial: Kanban **no es solamente un tablero**. Combina principios, visualización, límites WIP, sistema Pull, políticas, feedback y mejora evolutiva.

▼

Métricas en los 3 enfoques (Tradicional / Lean / Agile)

Una **métrica de software** es una medida cuantitativa de una característica del proceso, proyecto o producto. **No se mide por medir**: debe responder preguntas útiles para tomar decisiones y mejorar.

1. Enfoque tradicional

Tipo	Qué mide
Proceso	Rendimiento y características del proceso de desarrollo.
Proyecto	Estado, cumplimiento de planes y resultados de un proyecto.
Producto	Características del software obtenido.

Cuatro medidas básicas: tamaño, esfuerzo, tiempo calendario y defectos.

- **Tamaño:** dimensión del producto (por ejemplo, líneas de código, funcionalidades).
- **Esfuerzo:** cantidad de trabajo, expresada en horas-persona.
- **Tiempo calendario:** duración efectivamente transcurrida.
- **Defectos:** cantidad y clasificación de problemas detectados.

Esfuerzo ≠ duración. Ejemplo: dos personas, 8 h/día durante 5 días → **80 horas-persona** de esfuerzo, pero **5 días laborales** de duración.

Las métricas pueden observarse a nivel del **desarrollador** (tareas, esfuerzo, defectos), del **equipo** (hitos, avances, pruebas) y de la **organización** (presupuesto, estimaciones, defectos después del release).

2. Enfoques ágiles

Velocidad (Velocity)

Cantidad de trabajo **terminado** durante un Sprint, generalmente expresada en **Story Points**.

$$\text{Velocidad} = \sum \text{SP de historias terminadas}$$

Si se planificaron 40 SP pero solo se terminaron 25 SP, la velocidad del Sprint es **25 SP**. No se cuentan historias incompletas.

Se usa para prever entregas en un **mismo equipo**, no para comparar productividad individual entre equipos.

Capacidad (Capacity)

Tiempo de trabajo disponible en la iteración:

$$\text{Capacidad} = \sum \text{Horas disponibles del equipo}$$

Ejemplo: tres integrantes disponibles 30 horas cada uno → **90 horas**. Considera feriados, ausencias, dedicación y otras tareas.

Velocidad ≠ capacidad: velocidad expresa trabajo completado (frecuentemente en SP); capacidad expresa tiempo disponible (frecuentemente en horas). **Story Points no son horas.**

RTF — Running Tested Features

Cantidad de **funcionalidades implementadas, probadas y funcionando**. Mide avance verificable, no cantidad de código o tareas comenzadas.

Ejemplo: había 2 funcionalidades operativas y se suman 3 más → RTF = **5**, siempre que las anteriores sigan funcionando.

3. Lean/Kanban: métricas de flujo

Métrica	Desde / hasta	Qué representa
Lead Time	Solicitud → entrega	Tiempo total de espera del cliente y ejecución.
Cycle Time	Comienzo del trabajo → finalización	Tiempo transcurrido dentro del proceso.
Touch Time	Suma de períodos de trabajo efectivo	Tiempo en que realmente se trabajó, sin esperas.

Ejemplo: solicitud día 1, comienzo día 4, entrega día 11, trabajo efectivo 4 días.

- **Lead Time = 10 días.**
- **Cycle Time = 7 días.**
- **Touch Time = 4 días.**

Habitualmente **Touch Time ≤ Cycle Time ≤ Lead Time**.

Eficiencia del ciclo de proceso

$$\text{Eficiencia} = \frac{\text{Touch Time}}{\text{Elapsed Time}} \times 100$$

Si el intervalo elegido es de 10 días y hubo 4 días de trabajo efectivo, **eficiencia = 40 %**. Se debe identificar **qué intervalo se toma como Elapsed Time** en cada ejercicio.

Idea: una entrega puede tardar muchos días no porque la ejecución sea lenta, sino por esperas, colas y bloqueos.

4. Métricas orientadas al servicio

Concepto	Significado
Expectativa de nivel de servicio	Lo que el cliente espera recibir.
Capacidad de nivel de servicio	Lo que el sistema efectivamente puede entregar.
Acuerdo de nivel de servicio	Compromiso convenido con el cliente.
Umbral de adecuación	Mínimo aceptable antes de considerar insatisfactorio el servicio.

5. Métricas de producto

- **Tamaño:** líneas de código, clases, requerimientos, funcionalidades, etc.
- **Cobertura:** porcentaje de elementos o comportamientos alcanzados por pruebas, según criterio elegido.
- **Defectos por severidad:** distribución de defectos según gravedad.
- **Densidad de defectos:** defectos / tamaño del producto (la unidad depende de cómo se mida el tamaño).

Para el parcial: distinguir **esfuerzo y duración**, **velocidad y capacidad**, **RTF**, calcular **Lead/Cycle/Touch Time y eficiencia** y explicar para qué se usan las métricas de cada enfoque.

▼

Unidad 3: Gestión del software como producto

Prácticas continuas

Integran **desarrollo, integración, verificación y entrega durante toda la evolución** del producto, en lugar de concentrar las pruebas y la integración al final.

Integración continua (CI)

Integrar cambios frecuentemente en una versión compartida y verificar su compatibilidad mediante **compilación y pruebas automatizadas**.

Flujo: cambios → integración al repositorio → build → ejecución de pruebas → feedback.

Se apoya en **SCM**: repositorio, commits, ramas, merge y control de versiones.

Entrega continua vs. despliegue continuo

Práctica	Idea central
Integración continua (CI)	Integrar y verificar cambios frecuentemente.

Práctica	Idea central
Continuous Delivery	Mantener una versión verificada y lista para desplegar cuando se decida.
Continuous Deployment	Publicar automáticamente las versiones que cumplen las verificaciones.

Aclaración de fuentes: la diferencia entre Delivery y Deployment es una explicación complementaria; las PPT no la desarrollan formalmente.

TDD — Test Driven Development

Técnica de desarrollo en la que **se escribe la prueba antes del código**. Su ciclo es:

1. **RED:** escribir un test que falle.
2. **GREEN:** escribir el código mínimo para que pase.
3. **REFACTOR:** mejorar la estructura sin cambiar el comportamiento externo.

Tres leyes: no escribir código de producción sin una prueba que falla; no escribir más prueba de la necesaria para producir el fallo; no escribir más código de producción del necesario para hacerla pasar.

Ejemplo: escribir primero una prueba donde sumar(2, 3) = 5; comprobar que falla; implementar lo mínimo; refactorizar y volver a probar.

Refactoring

Mejorar la estructura interna sin modificar el comportamiento externo. Ejemplo: eliminar código duplicado mediante una función reutilizable. Se ejecutan las pruebas antes y después de los cambios.

Refactorizar no significa incorporar funcionalidades nuevas.

ATDD — Acceptance Test Driven Development

Acordar **pruebas de aceptación y comportamientos esperados antes de desarrollar**, con perspectiva de negocio.

TDD	ATDD
Guía diseño e implementación del código.	Guía el comportamiento esperado por el cliente.

TDD	ATDD
Principalmente pruebas unitarias.	Pruebas de aceptación, criterios de aceptación y reglas de negocio.
Enfoque técnico.	Colaboración entre negocio, desarrolladores y testers.

Testing continuo, automatización y regresión

Se prueba durante el desarrollo para obtener **feedback temprano**.

- **Automatizar** permite ejecutar verificaciones frecuentemente.
- **Regresión** comprueba que los cambios no afecten funciones existentes.
- Las **pruebas se versionan junto con el código**.

Para el parcial: explicar **CI**, diferenciar **Delivery/Deployment**, conocer **Red–Green–Refactor**, las **tres leyes de TDD**, **refactoring** y **TDD vs. ATDD**.

▼

Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

▼ Conceptos generales sobre calidad

Calidad: características de un producto o servicio relacionadas con su capacidad de satisfacer **necesidades manifiestas o implícitas**. Es relativa al contexto, el usuario y el tiempo. Un producto de calidad debe satisfacer al cliente, al usuario y a otros interesados.

Cinco visiones de la calidad

Visión	Qué considera
Del usuario	Satisfacción de necesidades y expectativas.
Del producto	Atributos y características del producto.
De manufactura	Cumplimiento de estándares y especificaciones.
Basada en el valor	Relación entre calidad, utilidad y costo.
Trascendental	Excelencia reconocible, difícil de definir o medir.

Calidad programada, realizada y necesaria

- **Programada:** lo planificado.
- **Realizada:** lo efectivamente construido.
- **Necesaria:** lo que realmente necesita el usuario.

La gestión de calidad busca hacer coincidir las tres. Lo que queda fuera de su coincidencia puede producir **desperdicio o insatisfacción**.

Calidad de proceso vs. calidad de producto

Calidad de proceso	Calidad de producto
Cómo se desarrolla el software.	**Qué características tiene** el software obtenido.
Procesos, prácticas, métodos, estándares.	Funcionalidad, rendimiento, seguridad, mantenibilidad, etc.
Previene problemas y mejora cómo se trabaja.	Evalúa si el producto cumple lo esperado.

Proceso de software: conjunto de actividades, prácticas y métodos utilizados para desarrollar o mantener software. Para definirlo deben establecerse **quién, qué, cuándo y cómo** realiza cada actividad.

Idea clave: mejorar el proceso favorece la calidad del producto, aunque no la garantiza por sí solo.

▼

Importancia de trabajar para y con Calidad. Ventajas y Desventajas

3.4. Importancia de trabajar con calidad

La presentación señala problemas habituales en proyectos de software:

- Atrasos en las entregas.
- Costos excedidos.
- Incumplimiento de compromisos.
- Requerimientos poco claros.
- Software que no cumple lo esperado.
- Horas extra y retrabajo.

- Dificultades para localizar o mantener componentes.

Trabajar con calidad ayuda a prevenir estos problemas mediante procesos definidos, controles, evaluaciones y mejora continua.

Principios de calidad según la cátedra

La diapositiva 8 presenta los siguientes principios:

1. La calidad no se inyecta ni se compra: debe estar incorporada desde el comienzo.
2. Es un esfuerzo de todos los integrantes.
3. Las personas son fundamentales y deben capacitarse.
4. Se necesita apoyo gerencial.
5. Hay que liderar mediante el ejemplo.
6. No se puede controlar lo que no se mide.
7. Se debe comenzar por lo básico, con simplicidad.
8. El aseguramiento de calidad debe planificarse.
9. Aumentar las pruebas no significa automáticamente aumentar la calidad.
10. Las prácticas de calidad deben ser razonables para el negocio.

Ventajas de trabajar con calidad

Permite reducir defectos y retrabajo, mejorar la previsibilidad, satisfacer a los clientes y facilitar el mantenimiento del software.

También favorece el control de los procesos y la posibilidad de repetir buenas prácticas.

Desventajas - Costos y dificultades

Implementar prácticas de calidad requiere capacitación, tiempo, herramientas, planificación y participación de los integrantes.

Las revisiones y controles también consumen recursos.

La cátedra destaca que estos esfuerzos pueden aumentar inicialmente el trabajo, pero los beneficios de prevenir defectos y reducir el retrabajo superan esos costos.

Su conclusión es que **trabajar con calidad resulta más barato que desarrollar sin calidad y corregir después los problemas**.

▼

Actividades relacionadas con el Aseguramiento de la Calidad del Software

La **Administración de Calidad del Software** define y controla mecanismos para alcanzar los niveles de calidad requeridos. Debe fomentar una **cultura de calidad compartida**, no limitada al grupo de QA.

Aseguramiento, planificación y control

Actividad	Función
Aseguramiento de calidad (SQA)	Establecer estándares, procedimientos y métodos organizacionales.
Planificación de calidad	Seleccionar y adaptar estándares y procedimientos al proyecto.
Control de calidad	Verificar el cumplimiento de estándares y procedimientos mediante revisiones, evaluaciones y mediciones.

Ejemplo: se exige revisar requerimientos. Aseguramiento define el procedimiento, planificación decide cómo aplicarlo en el proyecto y control comprueba que se realizó.

Funciones del SQA

Evaluar planificación, requerimientos, diseño, programación, integración y pruebas; establecer técnicas, estándares y herramientas; identificar y reportar desviaciones y adaptar prácticas al proyecto.

Grupo de Aseguramiento de Calidad (GAC)

Debe conservar **independencia** para evaluar objetivamente los procesos y reportar problemas. La PPT indica que **no debería reportar al Gerente de Proyectos** y debería tener acceso cercano a la gerencia superior.

Estándares y Plan de Calidad

- **Estándar de producto:** define cómo debe ser un producto (ejemplo: reglas de codificación).

- **Estándar de proceso:** define cómo debe realizarse una actividad (ejemplo: revisión de requerimientos).
- **Plan de Calidad:** establece atributos y niveles deseados, estándares aplicables y **cómo serán evaluados**.

No se aplican estándares inapropiados solo por estar definidos.

Ciclo de mejora

Definir procesos → desarrollar producto → evaluar calidad.

- Si **no** cumple: mejorar procesos y revisar de nuevo.
- Si **sí** cumple: estandarizar las prácticas útiles.

Para el parcial: distinguir **SQA, planificación y control**, funciones e **independencia del GAC**, estándar de producto/proceso y contenido del **Plan de Calidad**.

▼

Principales Modelos de Calidad existentes (CMMI – SPICE)

Los modelos de calidad y mejora aportan referencias para **evaluar y mejorar procesos** y comprender su nivel de madurez o capacidad.

CMMI — Capability Maturity Model Integration

Modelo de referencia para **evaluar y mejorar los procesos** de una organización, evolución del SW-CMM y asociado históricamente al SEI. En la presentación se aclara que **no es una norma ISO** y su evaluación corresponde a profesionales habilitados.

Constelaciones de la PPT:

- **CMMI-DEV:** desarrollo de productos y servicios.
- **CMMI-SVC:** gestión y entrega de servicios.
- **CMMI-ACQ:** adquisición de productos y servicios.

Representaciones de CMMI

Por etapas	Continua
Evalúa la **madurez organizacional**.	Evalúa la **capacidad de cada área de proceso**.
Cinco niveles **1–5**.	Seis niveles **0–5**.
Clasificación global y secuencia de mejora.	Permite priorizar áreas específicas.

Cinco niveles de madurez

Nivel	Nombre	Característica
1	Inicial	Procesos impredecibles, reactivos y poco controlados.
2	Administrado	Procesos gestionados **por proyecto**; incluye **PPQA**, requerimientos y configuración.
3	Definido	Procesos establecidos y estandarizados **en la organización**.
4	Cuantitativamente Administrado	Procesos medidos y controlados con datos.
5	Optimizado	Mejora continua y análisis de causas raíz.

A mayor madurez, se busca mayor control y previsibilidad de los procesos.

SPICE

Software Process Improvement and Capability Evaluation: marco para **evaluación de capacidad y mejora de procesos**. La PPT lo menciona, pero no desarrolla sus niveles con el mismo detalle que CMMI.

Modelo IDEAL

Cinco etapas de mejora de procesos:

1. **Initiating:** iniciar y obtener respaldo.
2. **Diagnosing:** evaluar la situación actual.
3. **Establishing:** definir prioridades y planes.
4. **Acting:** ejecutar los cambios.

5. **Learning:** aprender de los resultados.

CMMI vs. Agile

CMMI	Agile
Mejora y estandarización de **procesos organizacionales**.	Adaptación, colaboración y respuesta a necesidades del cliente.
Busca previsibilidad y disciplina.	Busca feedback y entregas frecuentes.

No son necesariamente incompatibles: pueden complementarse según el contexto.

Nota de alcance: no se desarrollan las **normas ISO**, según la indicación anterior para este resumen.

Para el parcial: niveles **1–5** de CMMI, diferencia entre representación **por etapas y continua**, SPICE e **IDEAL**.

▼

Auditorías al Software — Puntos clave para el parcial

1. Los tres tipos de auditoría

Tipo	Qué evalúa
Auditoría de Proyecto	Verifica el **cumplimiento del proceso de desarrollo**, según lo establecido en el PACS (Plan de Aseguramiento de Calidad de Software). Comprueba la consistencia entre requerimientos, diseño, código y actividades de verificación y validación.
Auditoría de Configuración Funcional	Compara el software construido con los requerimientos de la ERS. Verifica que se implementen **solo y completamente** las funcionalidades especificadas y que la matriz de rastreabilidad esté actualizada.
Auditoría de Configuración Física	Comprueba que el software tal como está construido sea **consistente con su documentación técnica y de soporte**.

Para diferenciarlas:

- **Proyecto:** ¿se siguió el proceso de desarrollo definido?
- **Funcional:** ¿el producto cumple con los requerimientos?

- **Física:** ¿lo construido coincide con su documentación técnica?

2. Roles de una auditoría

Rol	Responsabilidades
Gerente de SQA	Prepara el plan de auditorías, calcula costos, asigna recursos y es responsable de resolver las no conformidades.
Auditor	Acuerda fecha y alcance, reúne y analiza evidencia objetiva, ejecuta la auditoría, elabora el informe y hace seguimiento de las acciones.
Auditado	Participa, presenta evidencia, responde al informe, propone el plan de acción y comunica su cumplimiento.

3. Las cuatro etapas del proceso de auditoría

1. **Preparación y planificación:** acordar fecha, lugar, objetivo, alcance y participantes; preparar documentación y checklist.
2. **Ejecución:** realizar la reunión de apertura, efectuar preguntas, solicitar evidencias y completar el checklist.
3. **Análisis y reporte del resultado:** evaluar hallazgos, preparar el informe preliminar, realizar la reunión de cierre y entregar el informe final.
4. **Seguimiento:** el auditado presenta un plan de acción para las desviaciones; el auditor revisa y verifica su cumplimiento y cierra las desviaciones corregidas.

Importante: la auditoría no termina al detectar una desviación; incluye el seguimiento de las acciones correctivas.

4. Diferencia entre desviación y observación

Resultado	Significado	¿Requiere plan de acción?
Desviación	Incumplimiento de un proceso, estándar o requerimiento documentado que puede afectar la conformidad o generar incertidumbre sobre la calidad.	**Sí.**

Resultado	Significado	¿Requiere plan de acción?
Observación	Condición o práctica que convendría mejorar y que puede convertirse en una desviación futura.	**No**, según la clasificación de la PPT.

Para el parcial: distinguir **los tres tipos de auditoría**, **sus tres roles**, **las cuatro etapas del proceso** y **desviación vs. observación**.

▼

Calidad de Producto: Planificación, niveles, tipos, técnicas y herramientas de Testing — Puntos clave para el parcial

Testing es el conjunto de actividades para evaluar el software y **detectar defectos**. Se habla de un enfoque *destructivo* porque busca encontrar problemas; **no prueba la ausencia de errores**.

Error, defecto y falla

Concepto	Significado
Error	Equivocación humana.
Defecto	Problema incorporado en un producto de trabajo.
Falla	Comportamiento incorrecto que se manifiesta durante la ejecución.

Severidad = gravedad del defecto (bloqueante, crítica, mayor, menor, cosmética); **prioridad** = urgencia de resolverlo (urgencia, alta, media, baja).

Cuatro niveles de Testing

Nivel	Qué se prueba	Ejemplo
Unitario	Una unidad aislada.	Función de descuento.
Integración	Interacción entre componentes.	Ventas actualiza inventario.
Sistema	Sistema completo; requisitos funcionales y no funcionales.	Compra de extremo a extremo.

Nivel	Qué se prueba	Ejemplo
Aceptación	Necesidades y expectativas del usuario.	Usuario valida el circuito de venta.

En integración pueden aplicarse enfoques **Top-down** y **Bottom-up**. En aceptación, **Alfa** corresponde a ambiente controlado y **Beta** a condiciones reales de utilización.

Proceso de Testing y casos de prueba

1. **Planificación y control:** objetivos, riesgos, recursos, estrategia y criterios de finalización en el **Plan de Testing**.
2. **Identificación y especificación:** condiciones, casos, datos y ambientes.
3. **Ejecución:** correr los casos, comparar resultados, registrar defectos.
4. **Evaluación y reporte:** analizar resultados e informar.

Un **caso de prueba** tiene **objetivo**, **datos de entrada y ambiente** y **comportamiento esperado**.

Ejemplo — inicio de sesión: usuario registrado y contraseña correcta → resultado esperado: ingreso permitido.

Caja Negra vs. Caja Blanca

Caja Negra	Caja Blanca
Basada en especificaciones, entradas y salidas.	Basada en código, rutas, condiciones y decisiones internas.
No requiere conocer la implementación.	Requiere conocimiento de la implementación.
Equivalencias y valores límite.	**Cobertura estructural**.

Partición de equivalencias

Agrupa entradas que deberían recibir **tratamiento equivalente**, con clases válidas e inválidas.

Ejemplo: cantidad permitida de **1 a 100**:

- **CE inválida:** valores menores que 1 → probar **0**.
- **CE válida:** entre 1 y 100 → probar **50**.

- **CE inválida:** mayores que 100 → probar **101**.

Análisis de valores límite

Se prueban los **bordes** de las clases y sus valores inmediatamente adyacentes.

Para el rango **1–100**, probar **0, 1, 100, 101**.

Equivalencias: representantes de clases. **Valores límite:** extremos de las clases.

Técnicas basadas en experiencia

- **Adivinanza de defectos:** anticipar errores usando experiencia (campos vacíos, duplicados, caracteres especiales).
- **Testing exploratorio:** aprender y decidir nuevas pruebas mientras se explora el sistema, **no es probar al azar**.

Siete principios del Testing

1. Muestra **presencia de defectos**, no su ausencia.
2. **Testing exhaustivo imposible.**
3. **Testing temprano** reduce retrabajo.
4. Los defectos suelen **agruparse** en ciertos componentes.
5. **Paradoja del pesticida:** repetir siempre las mismas pruebas pierde efectividad.
6. Testing **depende del contexto**.
7. **Falacia de ausencia de errores:** sin defectos detectados no significa útil para el usuario.

Funcionales, no funcionales, regresión y Smoke

- **Funcionales:** qué hace el sistema (registrar, comprar, consultar).
- **No funcionales:** cómo se comporta (performance, carga, estrés, volumen, seguridad, instalación).
- **Regresión:** tras cambiar software, volver a probar que no se dañaron funciones existentes.

- **Smoke Test:** revisión básica inicial para detectar fallas graves antes de continuar.

Para el parcial: niveles y tipos, proceso y casos, **Caja Negra**, resolver **equivalencias y valores límite**, los **siete principios**.

▼

Técnicas y Herramientas para la realización de revisiones técnicas del software — Puntos clave para el parcial

7. Revisiones Técnicas del Software

1. ¿Qué son y para qué sirven?

Las **revisiones técnicas** son actividades de **Verificación y Validación (V&V) estáticas**: permiten identificar defectos en los productos de trabajo **sin ejecutar el software**.

Su objetivo principal es **detectar defectos lo más temprano posible**, antes de que se propaguen y generen retrabajo. Se pueden aplicar sobre requerimientos, diseños, código, planes de prueba y otros documentos o productos legibles del desarrollo.

Verificación: ¿estamos construyendo el producto correctamente, de acuerdo con sus especificaciones?

Validación: ¿estamos construyendo el producto correcto, de acuerdo con las necesidades del usuario?

2. Walkthrough vs. Inspección

Walkthrough (Recorrida)	Inspección
Informal; la dirige generalmente el autor del producto.	**Formal y sistemática**; la coordina un moderador.
Poca o ninguna preparación previa.	Requiere preparación individual anticipada.
Favorece el aprendizaje y el feedback rápido.	Busca identificar y remover defectos de manera eficiente.
No utiliza métricas formales ni un proceso controlado.	Utiliza **roles, checklists, registros y mediciones**.

Ejemplo de Walkthrough: un desarrollador presenta su diseño al equipo y recibe preguntas y comentarios sobre posibles problemas.

Ejemplo de Inspección: los inspectores revisan una especificación antes de una reunión formal, registran defectos y luego se verifica su corrección.

3. Los cinco roles de una Inspección

Rol	Responsabilidad
Autor	Crea o mantiene el producto inspeccionado; corrige los defectos e informa el retrabajo.
Moderador	Planifica, organiza y dirige la revisión; distribuye el material y hace seguimiento de los defectos.
Lector	Lee o presenta el producto durante la reunión.
Anotador	Registra los hallazgos de la revisión.
Inspector	Examina previamente el producto y detecta posibles defectos.

La PPT indica que el moderador debe proporcionar el producto a los inspectores **48 horas antes de la reunión**.

4. Las seis etapas del proceso de Inspección

1. **Planificación:** seleccionar el equipo, preparar el material y establecer el calendario.
2. **Visión general:** presentar el producto y el proceso de inspección.
3. **Preparación:** cada inspector analiza individualmente el producto para detectar defectos.
4. **Reunión de inspección:** compartir y registrar hallazgos, analizar posibles defectos y descartar falsos positivos.
5. **Corrección:** el autor corrige los defectos identificados.
6. **Seguimiento:** verificar que las correcciones estén realizadas y recolectar datos.

La reunión puede concluir con la **aceptación sin modificaciones**, el **rechazo** o la **aceptación provisional** del producto.

5. Reglas fundamentales

- **Revisar el producto, no al productor.** No se evalúa el desempeño de las personas.
- Establecer una agenda y respetarla; la PPT recomienda **reuniones de inspección de dos horas como máximo**.
- Registrar los defectos por escrito y utilizar **checklists**.
- Preparar anticipadamente el material y limitar el número de participantes.
- **Identificar los problemas, no intentar resolver todos durante la reunión.**
- Realizar seguimiento para comprobar que los defectos fueron corregidos.

Para el parcial: saber explicar el **propósito de las revisiones técnicas**, diferenciar **Walkthrough e Inspección**, y reconocer **los cinco roles y las seis etapas del proceso de Inspección**.

▼

Testing en ambientes Ágiles — Puntos clave para el parcial

El **Testing Ágil** se realiza **durante todo el desarrollo**, no solamente al final. Busca **feedback temprano**, prevención de defectos, automatización y **calidad compartida por todo el equipo**.

Manifiesto del Testing Ágil

Se valora más **probar durante el desarrollo** que al final; **prevenir** que solo encontrar defectos; **comprender** lo probado que verificar mecánicamente; **construir mejor software** que solo romperlo; y **responsabilidad de todos** que responsabilidad exclusiva del tester.

Nueve principios

1. Testing se mueve hacia adelante: **comenzar temprano**.
2. Testing **no es una fase**.
3. **Todos** participan en testing.
4. Reducir la **latencia del feedback**.

5. Las pruebas representan **expectativas**.
6. Mantener el código limpio y **corregir rápido**.
7. Reducir documentación innecesaria.
8. Las pruebas forman parte del **Done**.
9. Pasar de pruebas al final a **desarrollo guiado por pruebas**.

Seis prácticas

1. **Unitarias e integración automatizadas.**
2. **TDD** (Red–Green–Refactor, explicado en Unidad 3).
3. **Regresión automatizada de sistema.**
4. **ATDD** (aceptación acordada antes de implementar).
5. **Pruebas exploratorias.**
6. **Versionar las pruebas junto al código.**

Rol del tester

- **Con Product Owner:** clarificar historias y criterios de aceptación.
- **Con desarrolladores:** diseñar y automatizar pruebas; detectar defectos temprano.
- **Con todo el equipo:** definir Done, preparar ambientes, validar historias.

La calidad pertenece a todo el equipo, aunque el tester aporte conocimientos especializados.

Cuadrantes del Testing Ágil

Cuadrante	Orientación	Ejemplos
Q1	Tecnología; **guía el desarrollo**.	Unitarias/componentes.
Q2	Negocio; **guía el desarrollo**.	Criterios de aceptación, ejemplos y pruebas funcionales.
Q3	Negocio; **critica el producto**.	Exploratorias, usabilidad, aceptación.

Cuadrante	Orientación	Ejemplos
Q4	Tecnología; **critica el producto**.	Carga, seguridad, rendimiento, recuperación.

Q1 + Q2 guían; Q3 + Q4 critican. Q1 + Q4 tecnología; Q2 + Q3 negocio.

Pirámide tradicional vs. ágil

La PPT compara una pirámide **tradicional invertida**, con muchas pruebas de interfaz, frente a una **ágil normal**, con una base amplia de pruebas unitarias.

attachment:19ba968c-fa4e-4766-b92a-9863152e6d85:piramide_testing_tradicional_vs_agil.html

Distribución ilustrativa	Tradicional	Ágil
80–90 %	GUI / End-to-End	**Unitarias**
5–15 %	Integración	Aceptación
1–5 %	Unitarias	GUI

Las pruebas unitarias suelen ser **más rápidas y menos costosas de mantener**; las GUI siguen siendo necesarias, pero en general cuestan más. Estos porcentajes corresponden al **esquema de la cátedra**, no a reglas universales.

Beneficios de automatizar

Rapidez, reutilización, mayor cobertura, precisión, alcance y menores costos repetitivos. No reemplaza el criterio humano ni las pruebas exploratorias. Incorporar testing ágil también requiere un **cambio cultural**.

Para el parcial: nueve principios, seis prácticas, **cuadrantes Q1–Q4**, rol del tester y **comparación de las dos pirámides**.